

第 1 周实验报告（Shell / Git / LaTeX）

唐朝伟—学号：25020007111

2026 年 8 月 30 日

1 第 1 题含空格文件名的批量整理

```
mkdir q01 && cd q01
mkdir -p input/docs input/tmp
printf 'alpha\nbeta\n' > "input/docs/notes one.txt"
printf 'hidden\n' > input/docs/.secret.txt
touch input/tmp/empty.txt
printf '2026-08-24 09:00:01 start\n2026-08-24 09:00:02 done\n' > input/run.log

pwd
ls -laR input

mkdir -p "work/25020007111"
(cd input && find . -type f -name '*.txt' -exec cp --parents {} "../work/25020007111/"
\;)

find "work/25020007111" -type d -exec chmod 750 {} +
find "work/25020007111" -type f -exec chmod 640 {} +
(cd "work/25020007111" && find . -type f -printf '%P %s\n') > inventory.txt
```

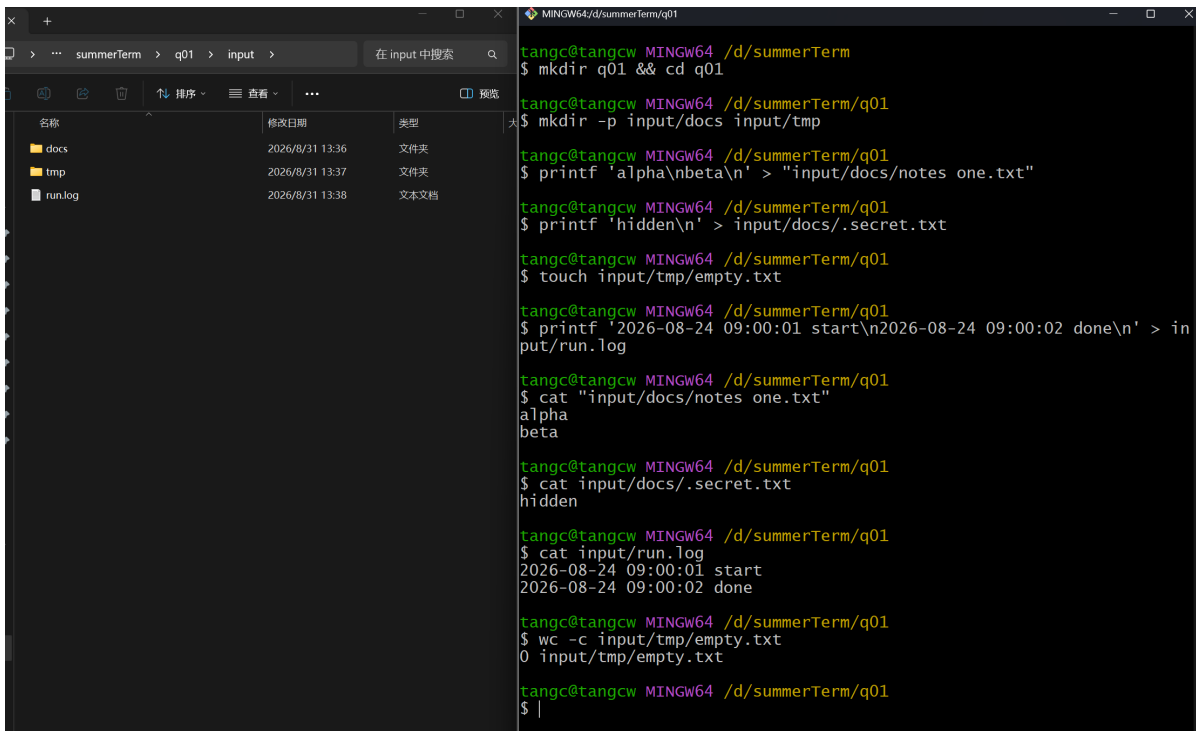


图 1: 第 1 题: 创建目录与文件, 查看内容并统计字节数

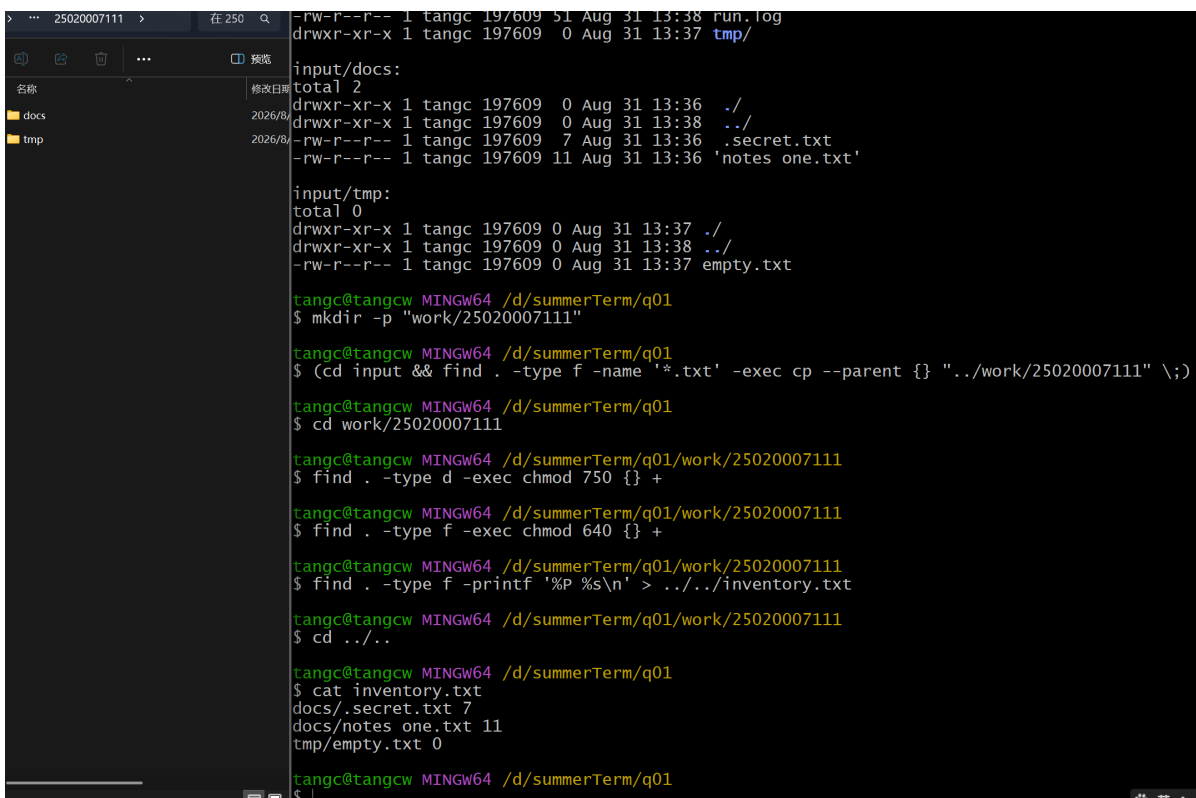


图 2: 第 1 题: 长格式列出含隐藏文件, 复制并改权限, 生成 inventory.txt

解题感悟

1. 文件名含空格：必须用引号包住整条路径；`find` 配合 `while read` 时加 `IFS=` 防去首尾空格、`-r` 防反斜杠转义。
2. 隐藏文件以点开头：shell 通配符 `*` 不匹配，但 `find -name '*.txt'` 会匹配到（所以复制时用 `find` 而非 `cp *.txt`）。
3. `mkdir -p` 递归建目录（父目录不存在则创建，否则忽略）；`touch` 建空文件，已存在只更新时间戳。
4. `cp --parents` 保留相对目录结构；权限 `chmod 750` 目录、`chmod 640` 文件；`find -printf '%P %s'` 输出相对路径 + 字节数。

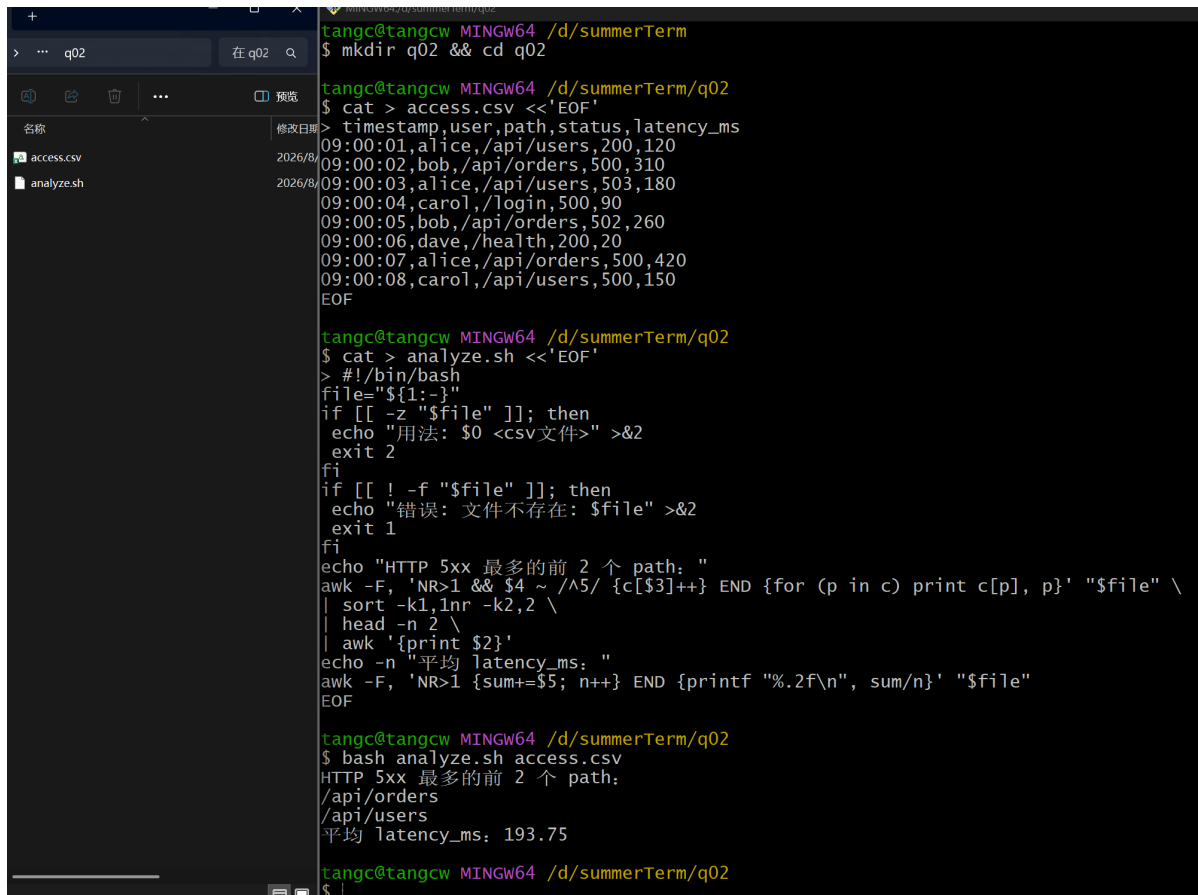
2 第 2 题随机访问日志统计

```
mkdir q02 && cd q02
cat > access.csv <<'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF

cat > analyze.sh <<'EOF'
#!/bin/bash
file="${1:-}"
if [[ -z "$file" ]]; then
    echo "用法: $0 <csv文件>" >&2
    exit 2
fi
if [[ ! -f "$file" ]]; then
    echo "错误: 文件不存在: $file" >&2
    exit 1
fi
echo "HTTP 5xx 最多的前 2 个 path: "
awk -F, 'NR>1 && $4 ~ /^5/ {c[$3]++} END {for (p in c) print c[p], p}' "$file" \
| sort -k1,1nr -k2,2 \
| head -n 2 \
| awk '{print $2}'
echo -n "平均 latency_ms: "
```

```
awk -F, 'NR>1 {sum+=$5; n++} END {printf "%.2f\n", sum/n}' "$file"
EOF

bash analyze.sh access.csv
bash analyze.sh no_such.csv; echo "退出码: $?"
```



```
tangc@tangcw MINGW64 /d/summerTerm
$ mkdir q02 && cd q02

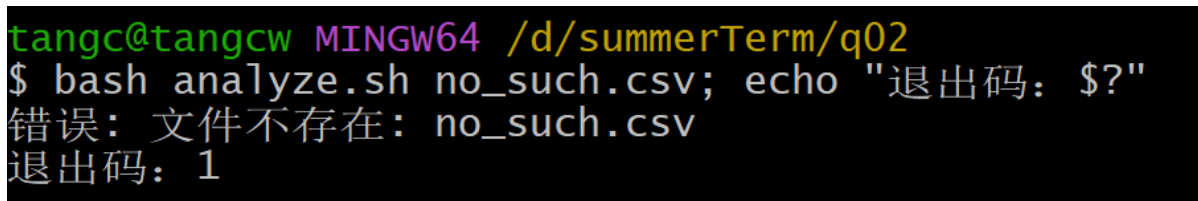
tangc@tangcw MINGW64 /d/summerTerm/q02
$ cat > access.csv <<'EOF'
> timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF

tangc@tangcw MINGW64 /d/summerTerm/q02
$ cat > analyze.sh <<'EOF'
> #!/bin/bash
file="${1:-}"
if [[ -z "$file" ]]; then
    echo "用法: $0 <csv文件>" >&2
    exit 2
fi
if [[ ! -f "$file" ]]; then
    echo "错误: 文件不存在: $file" >&2
    exit 1
fi
echo "HTTP 5xx 最多的前 2 个 path: "
awk -F, 'NR>1 && $4 ~ /^5/ {c[$3]++} END {for (p in c) print c[p], p}' "$file" \
| sort -k1,1nr -k2,2 \
| head -n 2 \
| awk '{print $2}'
echo -n "平均 latency_ms: "
awk -F, 'NR>1 {sum+=$5; n++} END {printf "%.2f\n", sum/n}' "$file"
EOF

tangc@tangcw MINGW64 /d/summerTerm/q02
$ bash analyze.sh access.csv
HTTP 5xx 最多的前 2 个 path:
/api/orders
/api/users
平均 latency_ms: 193.75

tangc@tangcw MINGW64 /d/summerTerm/q02
$
```

图 3: 第 2 题: 运行 analyze.sh 得到前 2 个 5xx path 与平均延迟 193.75



```
tangc@tangcw MINGW64 /d/summerTerm/q02
$ bash analyze.sh no_such.csv; echo "退出码: $?"
错误: 文件不存在: no_such.csv
退出码: 1
```

图 4: 第 2 题: 对不存在的文件运行, 错误写 stderr 且退出码为 1

解题感悟

1. 用 `cat > f <<'EOF'` 写多行文件且是 LF 换行; 别用 Windows 记事本 (CRLF, `awk` 取字段会多出 `\r`)。
2. 统计 5xx path: `awk -F, 'NR>1 && $4 ~ /^5/ {c[$3]++}'`, `NR>1` 跳过表头, `c[$3]++` 以 path 计数。

3. `sort -k1,1nr -k2,2` 数值逆序、同数按字典序；`head -n 2` 取前二。
4. 平均延迟: `awk 'NR>1 {sum+=$5; n++} END {printf "%.2f", sum/n}'`, 用 `printf` 保留两位小数。

3 第 3 题制造、解决并解释一次合并冲突

```
mkdir q03 && cd q03
git init -b main
git config user.name "sio"
git config user.email "tangcwyx@163.com"

echo "mode=normal" > config.txt
git add config.txt
git commit -m "initial commit"

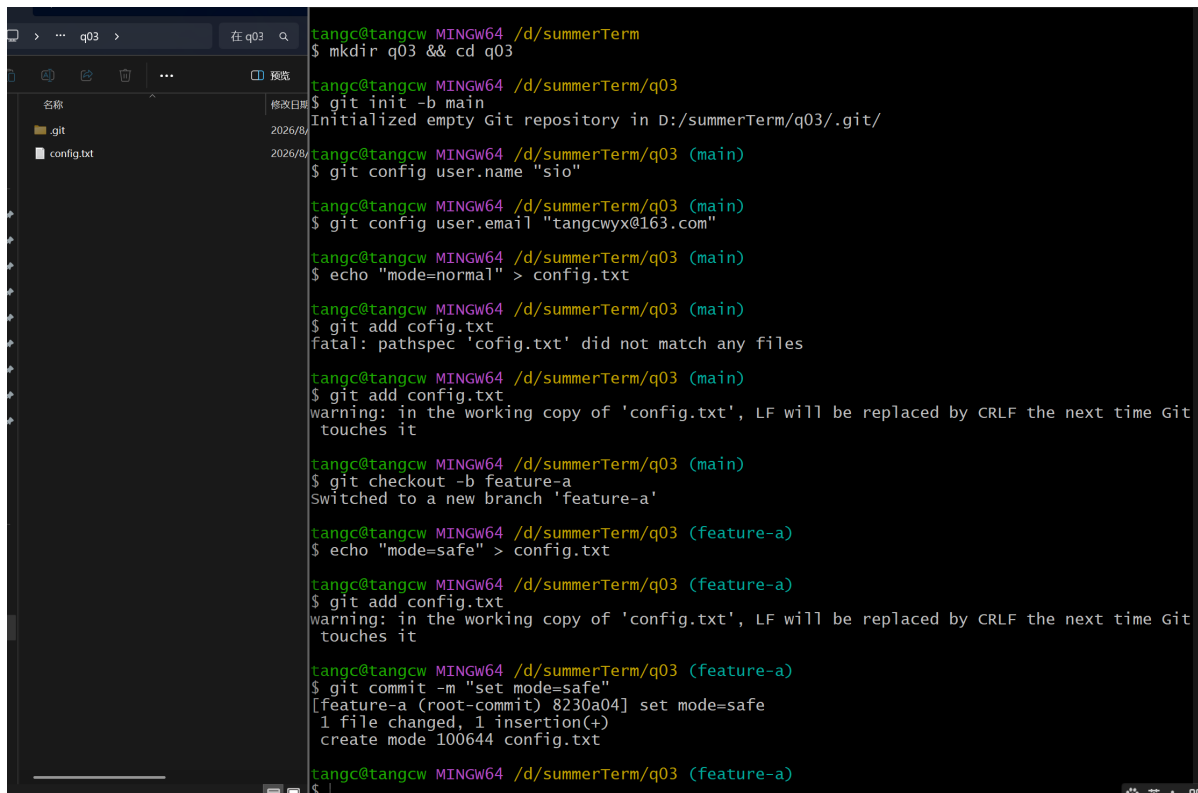
git checkout -b feature-a
echo "mode=safe" > config.txt
git add config.txt
git commit -m "set mode=safe"

git checkout main
git checkout -b feature-b
echo "mode=fast" > config.txt
git add config.txt
git commit -m "set mode=fast"

git checkout main
git merge feature-a
git merge feature-b    # 冲突

printf 'mode=safe\nnote=reviewed\n' > config.txt
git add config.txt
git commit -m "merge feature-b, keep mode=safe"

git status
git log --all --graph --decorate --oneline
```



The screenshot shows a terminal window with the following commands and output:

```
tangc@tangcw MINGW64 /d/summerTerm
$ mkdir q03 && cd q03

tangc@tangcw MINGW64 /d/summerTerm/q03
$ git init -b main
Initialized empty Git repository in D:/summerTerm/q03/.git/

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git config user.name "sio"

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git config user.email "tangcwyx@163.com"

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ echo "mode=normal" > config.txt

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git add config.txt
fatal: pathspec 'config.txt' did not match any files

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git checkout -b feature-a
Switched to a new branch 'feature-a'

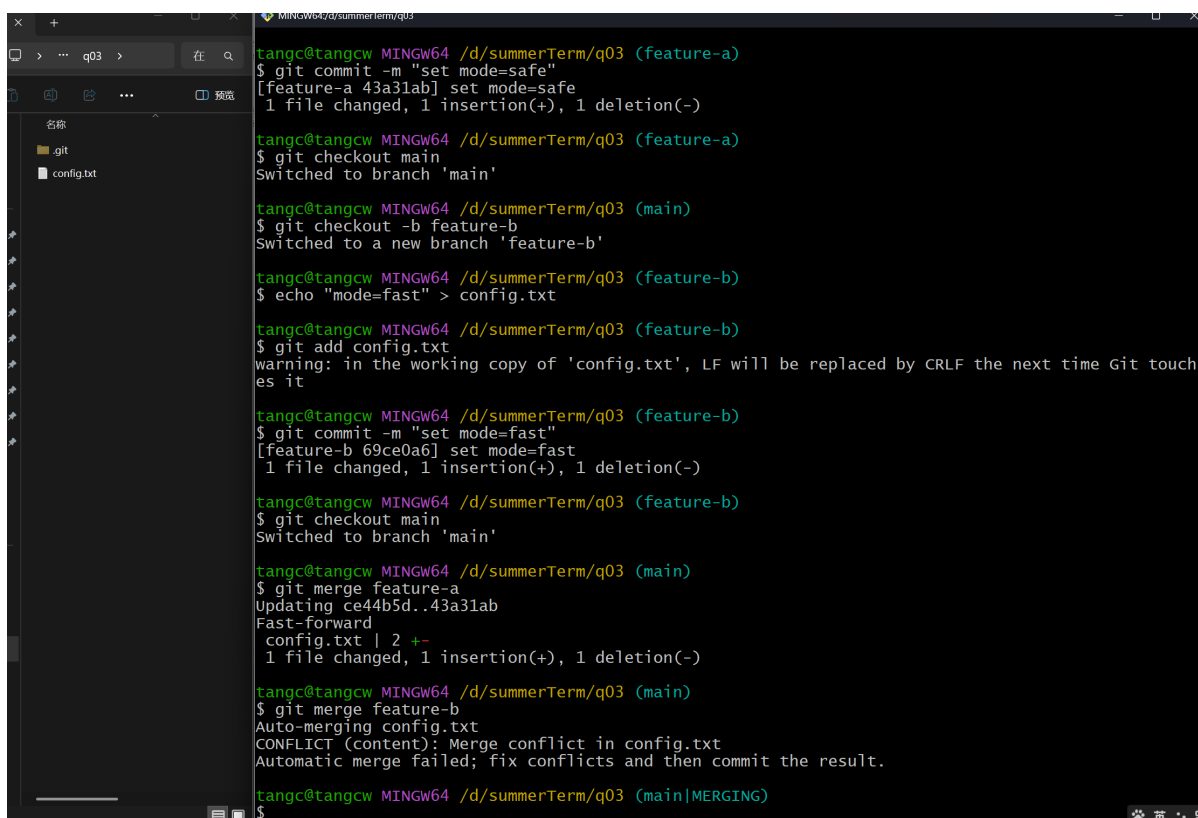
tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-a)
$ echo "mode=safe" > config.txt

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-a)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-a)
$ git commit -m "set mode=safe"
[feature-a (root-commit) 8230a04] set mode=safe
1 file changed, 1 insertion(+)
create mode 100644 config.txt

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-a)
$
```

图 5: 第 3 题: 初始化仓库并创建 feature-a 分支, 提交 mode=safe



The screenshot shows a terminal window with the following commands and output:

```
tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-a)
$ git commit -m "set mode=safe"
[feature-a 43a31ab] set mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-a)
$ git checkout main
Switched to branch 'main'

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git checkout -b feature-b
Switched to a new branch 'feature-b'

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-b)
$ echo "mode=fast" > config.txt

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-b)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-b)
$ git commit -m "set mode=fast"
[feature-b 69ce0a6] set mode=fast
1 file changed, 1 insertion(+), 1 deletion(-)

tangc@tangcw MINGW64 /d/summerTerm/q03 (feature-b)
$ git checkout main
Switched to branch 'main'

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git merge feature-a
Updating ce44b5d..43a31ab
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.

tangc@tangcw MINGW64 /d/summerTerm/q03 (main|MERGING)
$
```

图 6: 第 3 题: 合并 feature-a 后合并 feature-b 产生冲突, 进入 MERGING 状态

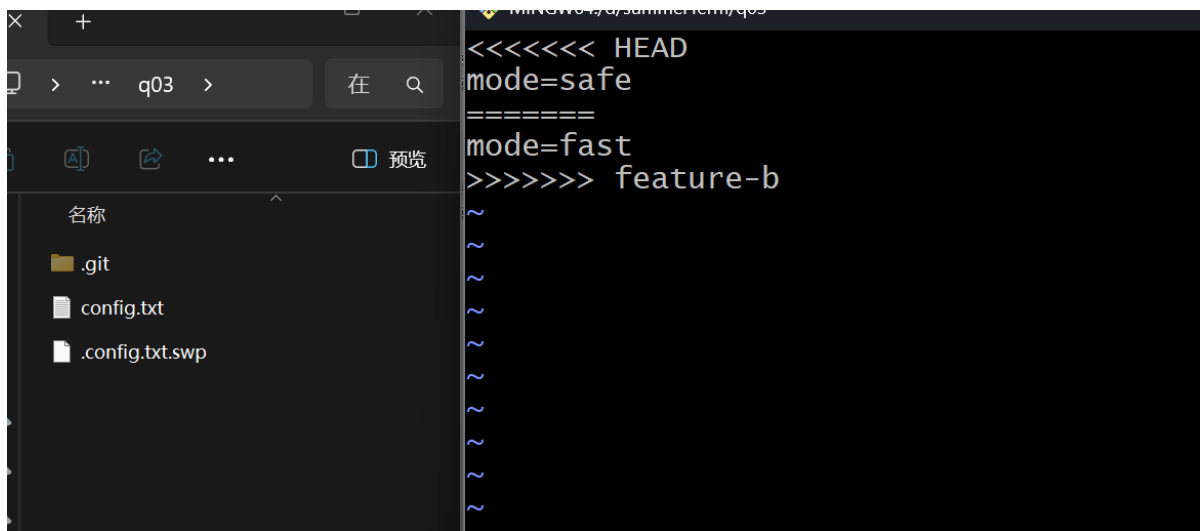


图 7: 第 3 题: config.txt 中的冲突标记 «««< / ===== / »»»>



MINGW64:/d/summerTerm/q03

```
mode=safe
```

```
note=reviewed
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

```
2
```

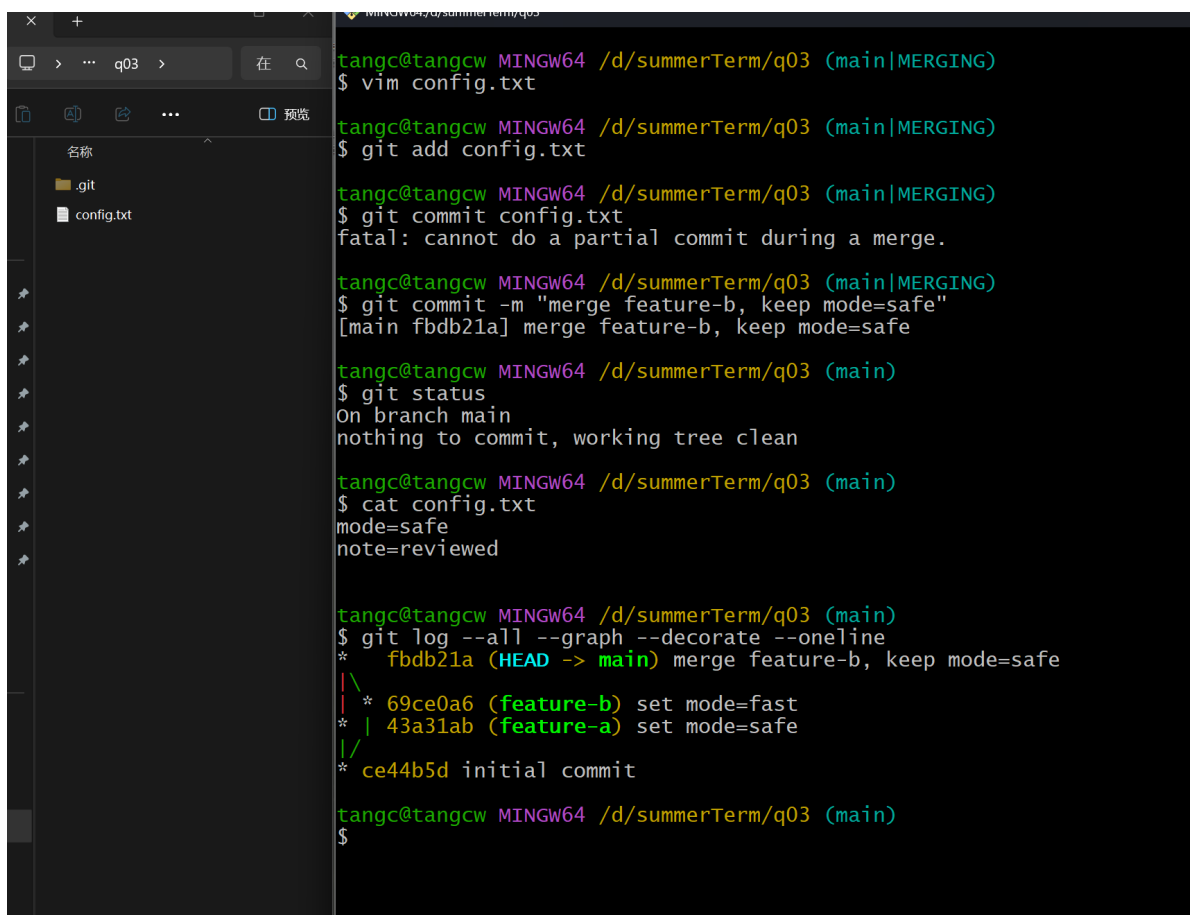
```
2
```

```
2
```

```
2
```

```
2
```

```
2
```



```
tangc@tangcw MINGW64 /d/summerTerm/q03 (main|MERGING)
$ vim config.txt

tangc@tangcw MINGW64 /d/summerTerm/q03 (main|MERGING)
$ git add config.txt

tangc@tangcw MINGW64 /d/summerTerm/q03 (main|MERGING)
$ git commit config.txt
fatal: cannot do a partial commit during a merge.

tangc@tangcw MINGW64 /d/summerTerm/q03 (main|MERGING)
$ git commit -m "merge feature-b, keep mode=safe"
[main fbdb21a] merge feature-b, keep mode=safe

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git status
On branch main
nothing to commit, working tree clean

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ cat config.txt
mode=safe
note=reviewed

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$ git log --all --graph --decorate --oneline
* fbdb21a (HEAD -> main) merge feature-b, keep mode=safe
|
| * 69ce0a6 (feature-b) set mode=fast
| * | 43a31ab (feature-a) set mode=safe
|/
* ce44b5d initial commit

tangc@tangcw MINGW64 /d/summerTerm/q03 (main)
$
```

图 9: 第 3 题: git add + commit 完成合并, 工作区干净, 分支图汇入 main

解题感悟

1. 三区模型: 工作区 (文件) → 暂存区 (git add) → 仓库 (git commit)。
2. git init -b main 初始化并指定主分支名 (-b = branch, 不指定则用默认分支名)。
3. 冲突成因: 两个分支改了同一文件的同一行, git 无法自动判断保留谁; 标记为 <<<<<<< /
===== / >>>>>>>。
4. 解决后 git add 加 git commit 完成合并; git log --all --graph --decorate --oneline 查看分支图。

4 第 4 题修复并构建一页技术说明

```
mkdir q04 && cd q04
export PATH="/c/texlive/2026/bin/windows:$PATH"
latexmk -pdf -halt-on-error report.tex
# 或 latexmk -pdf -xelatex -halt-on-error report.tex
```

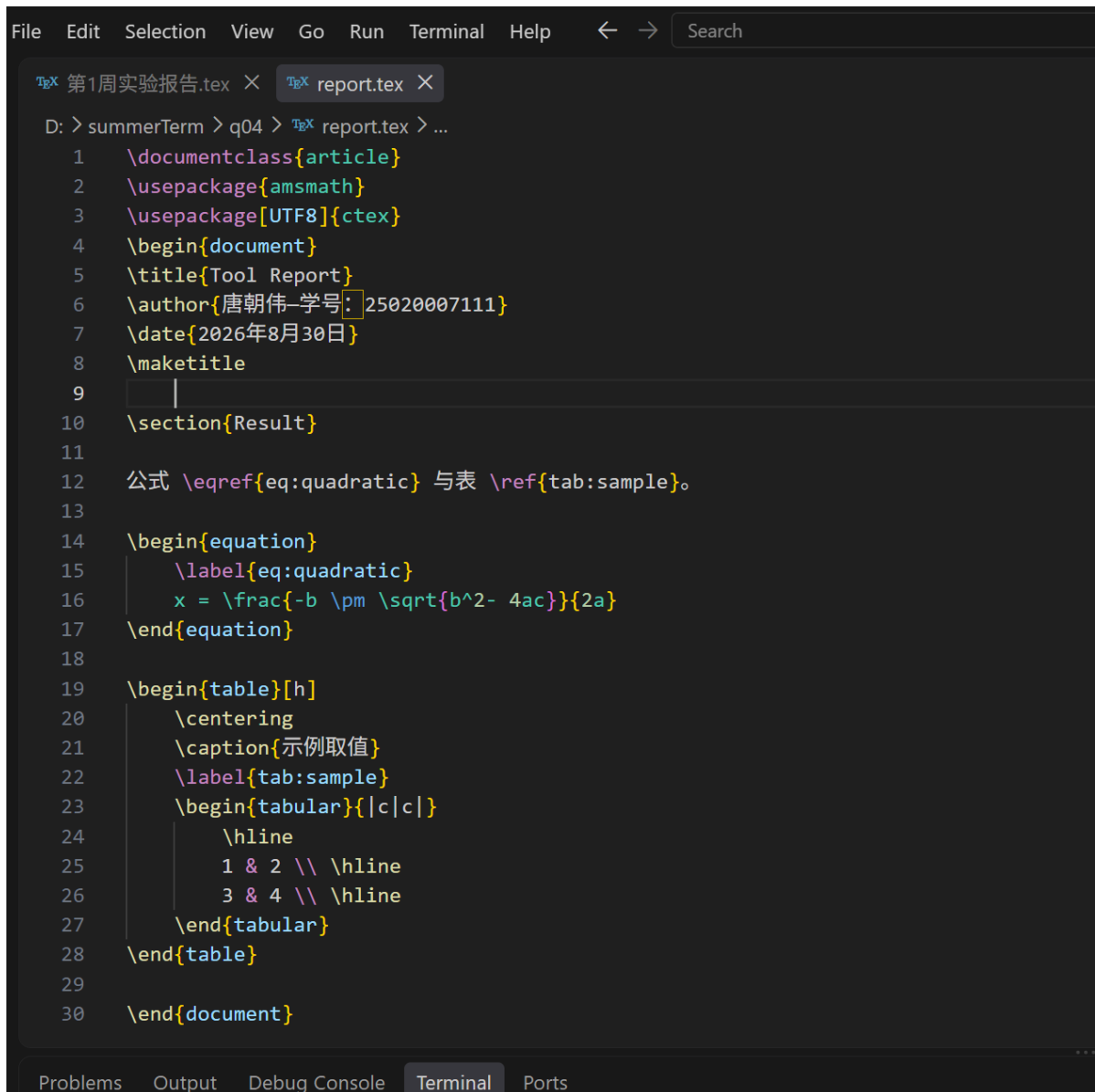
report.tex 关键片段:

```
\documentclass{article}
```

```

\usepackage{amsmath}
\usepackage[UTF8]{ctex}
\begin{document}
\title{Tool Report}\author{唐朝伟—学号：25020007111}\maketitle
\section{Result}
公式 \eqref{eq:quadratic} 与表 \ref{tab:sample}。
\begin{equation}\label{eq:quadratic}
x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}
\end{equation}
\begin{table}[h]\centering
\caption{示例取值}\label{tab:sample}
\begin{tabular}{|c|c|}
\hline
1 & 2 \\ \hline
3 & 4 \\ \hline
\end{tabular}
\end{table}
\end{document}

```



```
File Edit Selection View Go Run Terminal Help ← → Search
TeX 第1周实验报告.tex × TeX report.tex ×
D: > summerTerm > q04 > TeX report.tex > ...
1 \documentclass{article}
2 \usepackage{amsmath}
3 \usepackage[UTF8]{ctex}
4 \begin{document}
5 \title{Tool Report}
6 \author{唐朝伟-学号: 25020007111}
7 \date{2026年8月30日}
8 \maketitle
9 
10 \section{Result}
11 
12 公式 \eqref{eq:quadratic} 与表 \ref{tab:sample}。
13 
14 \begin{equation}
15 \quad \label{eq:quadratic}
16 \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}
17 \end{equation}
18 
19 \begin{table}[h]
20 \quad \centering
21 \quad \caption{示例取值}
22 \quad \label{tab:sample}
23 \quad \begin{tabular}{|c|c|}
24 \quad \quad \hline
25 \quad \quad 1 & 2 \quad \hline
26 \quad \quad 3 & 4 \quad \hline
27 \quad \quad \hline
28 \quad \end{tabular}
29 \end{table}
30 \end{document}
```

图 10: 第 4 题: report.tex 源码 (VSCode 编辑器)

Tool Report

唐朝伟一学号：25020007111

2026年8月30日

1 Result

公式 (1) 与表 1。

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (1)$$

表 1: 示例取值

1	2
3	4

图 11: 第 4 题：编译生成的 report.pdf，公式 (1) 与表 1 交叉引用正确，无??

解题感悟

1. 文档骨架：\documentclass 声明类型，\begin{document} 开始正文，之前是导言区。
2. 公式 equation 自动编号，\label 打标签，正文用\eqref 引用。

3. 表格: `table` (浮动体, `\caption` 标题、`\label` 标签) + `tabular` 画表 (& 分列、`\\` 换行、`\hline` 横线)。
4. 交叉引用依赖 `.aux` , 只编译一遍会显示?? ; 用 `latexmk -pdf -halt-on-error` 自动多遍编译解决。

5 课后练习 (missing-semester Shell 2~15)

5.1 练习 2: `ls -l` 与权限位

```
ls -l / # 前10位: 类型(1) + 属主(2-4) + 属组(5-7) + 其他(8-10)
```

1. `-l` (long format) 长格式: 每行显示权限、硬链接数、属主、属组、大小、修改时间、名字。
2. 前 10 位: 第 1 位文件类型 (- 普通、`d` 目录、`l` 链接等), 2 4 位属主、5 7 位属组、8 10 位其他用户, 权限为 `r` 读 / `w` 写 / `x` 执行。

5.2 练习 3: `glob` 通配符

```
ls *.txt file?.txt {a,b,c}.txt [ab].txt
```

1. `glob` 是 shell 用来匹配文件名的通配模式, 执行前由 shell 自身展开成一组已存在文件名; `find -name "*.zip` 里的要加引号交给 `find` 匹配, 而非让 shell 先展开。
2. `*` 匹配任意个字符 (不含路径分隔符)、`?` 恰好一个、`[ab]` 字符类匹配任一、`{a,b,c}` 花括号展开字符串组合 (不要求文件存在)。

```
MINGW64/d/summerTerm/tmp/g
tangc@tangcw MINGW64 /d/summerTerm
$ ls -l /
total 8220
-rw-r--r-- 1 tangc 197609 18765 Jul 22 12:53 LICENSE.txt
-rw-r--r-- 1 tangc 197609 311219 Jul 22 12:53 ReleaseNotes.html
drwxr-xr-x 1 tangc 197609 0 Aug 14 01:10 bin/
drwxr-xr-x 1 tangc 197609 0 Aug 14 01:10 cmd/
drwxr-xr-x 1 tangc 197609 0 Aug 14 01:10 dev/
drwxr-xr-x 1 tangc 197609 0 Aug 14 01:10 etc/
-rwxr-xr-x 1 tangc 197609 139080 Jul 22 12:26 git-bash.exe*
-rwxr-xr-x 1 tangc 197609 138568 Jul 22 12:26 git-cmd.exe*
drwxr-xr-x 1 tangc 197609 0 Aug 14 01:10 mingw64/
dr-xr-xr-x 9 tangc 197609 0 Sep 1 21:07 proc/
drwxr-xr-x 1 tangc 197609 0 Sep 1 21:06 tmp/
-rw-r--r-- 1 tangc 197609 1845074 Aug 14 01:10 unins000.dat
-rwxr-xr-x 1 tangc 197609 4444872 Aug 14 01:08 unins000.exe*
-rw-r--r-- 1 tangc 197609 25396 Aug 14 01:10 unins000.msg
drwxr-xr-x 1 tangc 197609 0 Aug 14 01:10 usr/

tangc@tangcw MINGW64 /d/summerTerm
$ mkdir -p tmp/g && cd tmp/g

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ touch a.txt b.txt c.txt file1.txt file2.txt file23.txt

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ ls *.txt
a.txt b.txt c.txt file1.txt file2.txt file23.txt

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ ls file?.txt
file1.txt file2.txt

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ ls {a,b,c}.txt
a.txt b.txt c.txt

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ ls [ab].txt
a.txt b.txt

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$
```

图 12: 课后练习: ls -l 权限位与 glob 通配符 (*/?/{a,b,c}/[ab])

5.3 练习 4: 三种引号

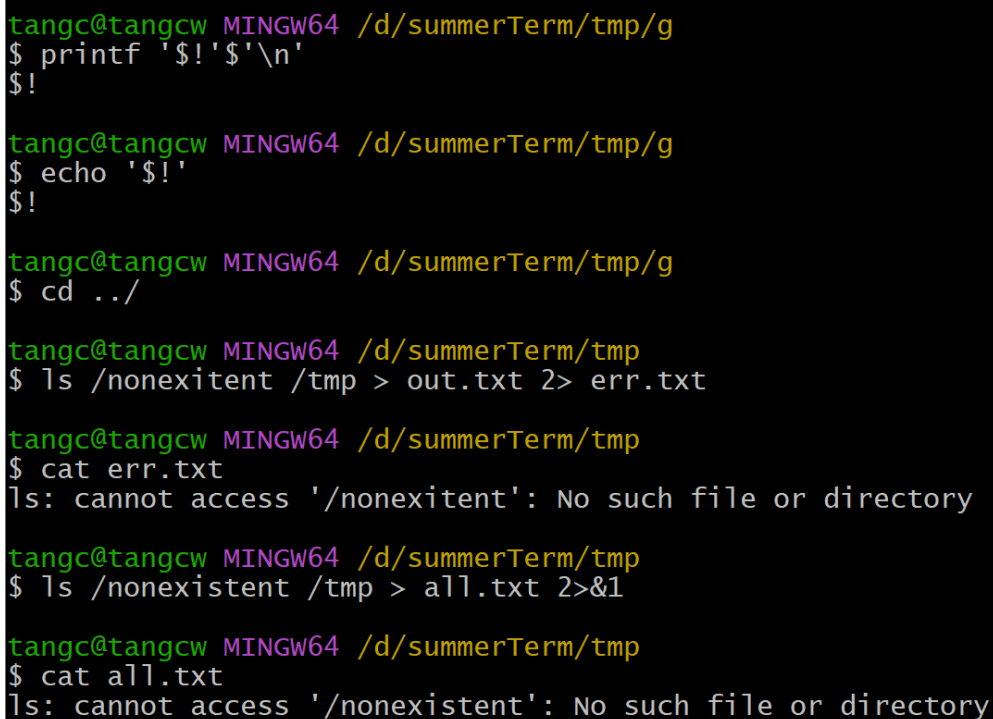
```
printf '$!$'\n'
```

1. '单引号'完全字面量 (\$、!、\ 均不解析); "双引号" 解析 \$、命令替换、部分\, ! 在交互式 bash 仍会历史展开。
2. '\$'...' (ANSI-C 引号) C 风格解析反斜杠转义: \n 换行、\t 制表符等。

5.4 练习 5: 标准流重定向

```
ls /nonexistent /tmp > out.txt 2> err.txt
ls /nonexistent /tmp > all.txt 2>&1
ls /nonexistent /tmp &> all.txt
```

1. 三条标准流: stdin(0)、stdout(1)、stderr(2); > 等价 1> 只重定向 stdout, 2> 重定向 stderr, < 指 stdin。
2. 2>&1 把 stderr 重定向到 stdout 当前指向处; &> 是 bash 简写, 等价 2>&1。
3. 顺序重要: > all.txt 2>&1 正确; 2>&1 > all.txt 会把 stderr 送到旧 stdout (屏幕)。



The screenshot shows a terminal window with the following commands and output:

```
tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ printf '$!'\n'
$!

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ echo '$!'
$!

tangc@tangcw MINGW64 /d/summerTerm/tmp/g
$ cd ../

tangc@tangcw MINGW64 /d/summerTerm/tmp
$ ls /nonexistent /tmp > out.txt 2> err.txt

tangc@tangcw MINGW64 /d/summerTerm/tmp
$ cat err.txt
ls: cannot access '/nonexistent': No such file or directory

tangc@tangcw MINGW64 /d/summerTerm/tmp
$ ls /nonexistent /tmp > all.txt 2>&1

tangc@tangcw MINGW64 /d/summerTerm/tmp
$ cat all.txt
ls: cannot access '/nonexistent': No such file or directory
```

图 13: 课后练习: 三种引号与标准流重定向 (`$$'\n'` 与 `2>1/2&1`)

5.5 练习 6: 退出状态与 && / ||

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

1. `$?` 保存上一条命令退出状态码 (0-255, 0 成功、1 一般错误、2 命令用法错误、255 越界)。
2. `&&` 前一条成功才执行后一条; `||` 前一条失败才执行后一条; `[ -d 路径 ]` 等价 `test -d 路径` 。

5.6 练习 7: 为什么 `cd` 是内建命令

因为子进程无法改变父进程的状态。若 `cd` 是独立程序, 它只会在自己的子进程里切换工作目录, 子进程退出后父进程 (shell) 的工作目录不变。而 `cd` 需要修改 shell 自身的当前工作目录, 所以必须由内建命令在当前进程内完成。同理 `export`、`alias` 也是内建命令。

5.7 练习 8：检查文件是否存在的脚本

```
if [ -f "$1" ]; then echo "$1 存在"; else echo "$1 不存在"; fi
```

1. `[ -f "$1" ]` 等价 `test -f "$1"`; `-f` 判断存在且是普通文件 (`-e` 只判断存在)。
2. `$1` 取第一个位置参数; 带引号的 `"$1"` 保留文件名的空格。

5.8 练习 9：chmod +x 与执行权限

```
./check.sh somefile      # Permission denied  
chmod +x check.sh  
./check.sh somefile
```

1. 直接 `./` 脚本执行需要脚本有可执行位 `x`; `chmod +x` 加上它。
2. 绕过方式: `bash check.sh somefile` (把脚本当参数交给 `bash` 解释, 不依赖 `x` 位)。
3. 在 Git Bash 或 WSL 的 `/mnt` 目录下 `chmod` 无效 (NTFS 无真 Unix 权限), 用 `bash` 运行时只用到 `r` 权限。

```
tangc@tangcw:~$ cd /mnt/d/summerTerm  
tangc@tangcw:/mnt/d/summerTerm$ cat > ex8.sh <<'EOF'  
> #!/bin/bash  
if [ -f "$1" ]; then  
    echo "$1 存在, 是普通文件"  
else  
    echo "$1 不存在 (或不是普通文件) "  
fi  
> EOF  
tangc@tangcw:/mnt/d/summerTerm$ bash ex8.sh nonexistent.txt  
nonexistent.txt 不存在 (或不是普通文件)  
tangc@tangcw:/mnt/d/summerTerm$ ls -l ex8.sh  
-rwxrwxrwx 1 tangc tangc 127 Sep  1 21:38 ex8.sh  
tangc@tangcw:/mnt/d/summerTerm$ chmod -x ex8.sh  
tangc@tangcw:/mnt/d/summerTerm$ ls -l ex8.sh  
-rwxrwxrwx 1 tangc tangc 127 Sep  1 21:38 ex8.sh  
tangc@tangcw:/mnt/d/summerTerm$ cp /mnt/d/summerTerm/ex8.sh ~/  
tangc@tangcw:/mnt/d/summerTerm$ cd ~/  
tangc@tangcw:~$ ls -l ex8.sh  
-rwxr-xr-x 1 tangc tangc 127 Sep  1 21:42 ex8.sh  
tangc@tangcw:~$ chmod -x ex8.sh  
tangc@tangcw:~$ ls -l ex8.sh  
-rw-r--r-- 1 tangc tangc 127 Sep  1 21:42 ex8.sh  
tangc@tangcw:~$ touch ex9.txt  
tangc@tangcw:~$ bash ex8.sh ex9.txt  
ex9.txt 存在, 是普通文件  
tangc@tangcw:~$ ./ex8.sh ex9.txt  
-bash: ./ex8.sh: Permission denied  
tangc@tangcw:~$
```

图 14: 课后练习：文件存在性脚本与 `chmod -x/+x` 执行权限变化

5.9 练习 10: set -x 跟踪

```
set -x # 每条命令执行前打印 (行首 +); set +x 关闭
```

1. `set -x` (xtrace) 开启命令跟踪: 每条命令执行前先打印到 stderr, 行首加 +, 便于调试; `set +x` 关闭。
2. `set` 选项: `-u` 变量未定义报错、`-e` 命令失败即退出、`-o pipefail` 管道任一段失败即失败。

```
tangc@tangcw:~$ cat > ex10.sh <<'EOF'
> #!/bin/bash
set -x
name="world"
echo "hello $name"
set +x
> EOF
tangc@tangcw:~$ ./ex10.sh
-bash: ./ex10.sh: Permission denied
tangc@tangcw:~$ bash ex10.sh
+ name=world
+ echo 'hello world'
hello world
+ set +x
```

图 15: 课后练习: `set -x` 命令跟踪输出

5.10 练习 11: 带日期的备份

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

1. `$(date +%Y-%m-%d)` 是命令替换: 执行 `date +%Y-%m-%d` 并用其输出 (如 2026-09-01) 替换到命令行。
2. `+` 是 `date` 命令的格式字符串前缀, `-` 是普通分隔符会被输出; `%` 后加 `-` 去掉前导零, `_` 用空格代替零, `^` 转大写。

```
tangc@tangcw:~$ touch notes.txt
tangc@tangcw:~$ cp notes.txt "notes_$(date +%Y-%m-%d).txt"
tangc@tangcw:~$ ls notes*.txt
notes.txt  notes_2026-09-01.txt
```

图 16: 课后练习: 命令替换 `$(date +%Y-%m-%d)` 生成日期备份名

5.11 练习 12: flaky test 参数化

```
while ! "$@"; do ;; done
```

1. "\$@" 展开为所有位置参数 (\$1 \$2 ...), 每个原样保留 (含空格), 因此能接收整条测试命令; 用"\$@" 而非 \$@, 避免参数被二次分词。
2. ! 对命令退出码取反: 测试失败则继续循环重试, 成功则退出; : 是空操作占位循环体。
3. \$# 参数个数、\$* (不加引号) 把所有参数拼成一个字符串。

5.12 练习 13: home 目录最常见的 5 种扩展名

```
find ~ -type f | grep -oE '\.[^./]+$' | sort | uniq -c | sort -nr | head -n 5
```

1. find ~ -type f 找 home 下所有普通文件; ~ 是当前用户家目录, 接用户名可指定用户。
2. grep -oE '\.[^./]+\$' 只输出行尾扩展名: \. 匹配字面点, [^./] 否定字符类 (排除点和斜杠), + 一次以上, \$ 行尾锚点。
3. sort 后 uniq -c 计数 (uniq 只统计相邻重复), sort -nr 按次数数值逆序, head -n 5 取前 5。

5.13 练习 14: xargs 统计.sh 行数

```
find . -type f -name '*.sh' -print0 | xargs -0 wc -l
```

1. find -print0 用 NUL 字符 (\0, 占 1 字节) 分隔文件名, 而不是默认换行, 文件名里的空格、换行不会被切碎。
2. xargs -0 按 NUL 分隔读取输入, 与 -print0 配套, 正确处理含空格文件名; xargs 把收到的文件名拼到 wc -l 后批量执行 (会自动按命令行长度上限分批)。

5.14 练习 15: curl 统计讲数

```
curl -s https://missing-semester-cn.github.io/2026/ \  
| grep -oE 'href="/2026/[a-z-]+/' | wc -l
```

1. curl -s 静默模式, 关闭进度输出; 课程页每讲链接的共性是 href="/2026/<slug>/"。
2. grep -oE 只输出匹配部分, wc -l 统计行数即讲数; 更简单可用 grep -c 'href="/2026/' 直接计数匹配行。

```

tangc@tangcw:~$ cd /mnt/c/Users/tangc/Downloads
tangc@tangcw:/mnt/c/Users/tangc/Downloads$ find ~ -type f | grep -oE '\.[^./]+$' | sort | uniq -c | sort -nr | head -n 5
 872 .pyc
 872 .py
 19 .typed
 18 .body
 13 .txt
tangc@tangcw:/mnt/c/Users/tangc/Downloads$ cd ~
tangc@tangcw:~$ find . -type f -name '*.sh' -print0 | xargs -0 wc -l
 5 ./ex10.sh
 6 ./ex8.sh
11 total
tangc@tangcw:~$ curl -s https://missing-semester-cn.github.io/2026/ | grep -oE 'href="/2026/[a-z-]+/' | wc -l
9
tangc@tangcw:~$

```

图 17: 课后练习：最常见扩展名、xargs 统计行数、curl 统计讲数

6 提交记录

课程要求将每次报告、练习、代码上传到个人 GitHub 并保留 commit 记录（禁止单次全量提交），故本报告采用分批提交：笔记 / 报告源码 / 报告 PDF / q01~q04 / 配置各占一次提交，共 8 次。

- 仓库链接: <https://github.com/silksnow/sys-dev-tools-course-2026>
- 提交记录截图:

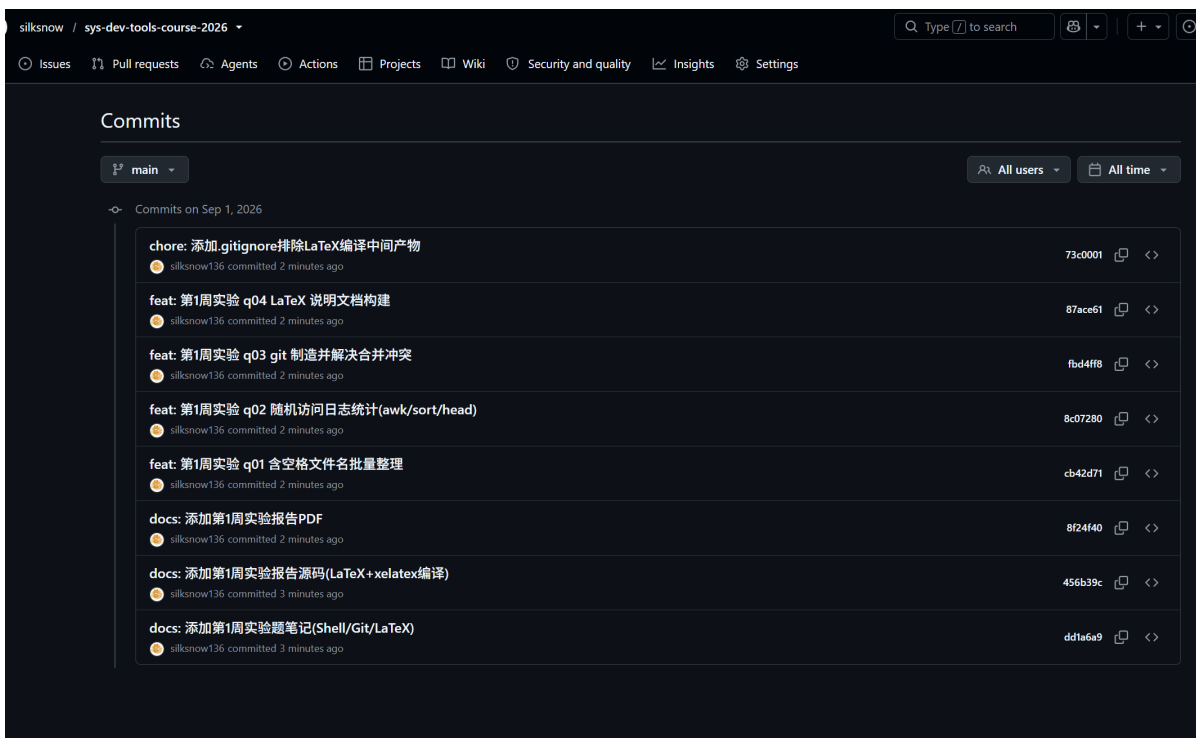


图 18: GitHub commit 记录（8 次分批提交）